Final Viva Guide

Everything in This Project, Explained in Simple Words

Phases 1, 2 and 3 — Model H (Hindi) and Model L (Nepali)

Swaraj Kumar
2025202016

Personal study document — not a graded deliverable

September 15, 2026

How to use this document

This is a study guide I wrote for myself so I can explain every part of this project confidently in the final viva. It is **not** a course deliverable and does not affect grading.
Every number in here comes from a file this project actually produced. Nothing is invented. Where something was *not* done, or was done wrongly and then fixed, this guide says so plainly — because an examiner will find those places anyway, and it is much better to walk in already knowing them.

The language is deliberately simple. Where a technical term appears, it is explained the first time in a green "In simple words" box.

If you only have one hour: read Chapter 1 (the cheat sheet), then Chapter 11 (the question bank), then Chapter 12 (mistakes to own). Everything else exists to make those three make sense.

Contents

1 The Cheat Sheet **5**
- 1.1 What I built, in three sentences . . . 5
- 1.2 The three findings I lead with . . . 5
- 1.3 Numbers to memorise . . . 6
- 1.4 The one thing I must never get wrong . . . 6

2 What This Project Is, In Plain Words **7**
- 2.1 The task . . . 7
- 2.2 Why two separate models and not one multilingual model . . . 7
- 2.3 The shape of the whole pipeline . . . 8

3 Phase 1 — Data and Tokenizers **9**
- 3.1 Where the text came from . . . 9
- 3.2 Cleaning — what I did and why, step by step . . . 9
  - 3.2.1 Step 1 — Unicode normalisation (NFC) . . . 9
  - 3.2.2 Step 2 — Strip pure-Latin tokens . . . 10
  - 3.2.3 Step 3 — Drop bad documents . . . 10
- 3.3 Splitting into train, validation and test . . . 11
- 3.4 Tokenizers . . . 11
  - 3.4.1 Why the vocabulary sizes differ . . . 12

4 Phase 2 — The Transformer, Piece by Piece **13**
- 4.1 The forward pass in tensor shapes . . . 13
- 4.2 Step 1 — Turning tokens into vectors . . . 14
- 4.3 Step 2 — Telling the model about position . . . 14
  - 4.3.1 What I used . . . 14
  - 4.3.2 RoPE — what it is and why I did not use it . . . 14
- 4.4 Step 3 — Multi-head causal self-attention . . . 15
  - 4.4.1 The three projections . . . 15
  - 4.4.2 Splitting into heads — the reshape . . . 16
  - 4.4.3 Why divide by $\sqrt{d_k}$ . . . 16
  - 4.4.4 The causal mask . . . 17
- 4.5 Step 4 — The rest of the block . . . 17
  - 4.5.1 Feed-forward network . . . 17
  - 4.5.2 Residual connections . . . 18
  - 4.5.3 Pre-norm vs post-norm . . . 18
  - 4.5.4 LayerNorm . . . 18

4.6 Step 5 — Output and weight tying . . . 18
4.7 The training objective . . . 18
4.8 Dropout — all four places . . . 19

5 Phase 2 — Training **20**
5.1 The settings, and why . . . 20
5.2 What actually happened . . . 21
5.3 Checkpoints and resume . . . 21

6 Phase 2 — Evaluation and What the Numbers Mean **23**
6.1 Perplexity and bits-per-byte — the headline . . . 23
6.2 Generation quality and the temperature question . . . 24
6.2.1 The subtle part — BLEU and chrF++ disagree . . . 25
6.3 Attention analysis . . . 26
6.3.1 Reading a heatmap out loud . . . 26

7 Phase 3 — The Reasoning Dataset **27**
7.1 What the assignment asked for . . . 27
7.2 The four task families . . . 27
7.3 Size and leakage control . . . 27
7.4 Two defects I caught in my own dataset . . . 28
7.4.1 A 45% gender-agreement leak . . . 28
7.4.2 Nepali numerals were completely unrepresentable . . . 29
7.5 Natural phrasing in Nepali . . . 29

8 Phase 3 — Finetuning **30**
8.1 The protocol . . . 30
8.2 Hyperparameters — identical for both models . . . 30
8.3 Two design decisions worth explaining . . . 31
8.3.1 Loss on the answer only . . . 31
8.3.2 Bucketing by prompt length instead of padding . . . 31
8.4 What happened during training . . . 31

9 Phase 3 — Results and What They Really Mean **33**
9.1 Why I report two metrics . . . 33
9.2 The headline table . . . 34
9.3 Per task family — the agreement with chance . . . 34
9.4 Why the two-hop task scores below chance . . . 34
9.5 One-hop answers show a recency bias . . . 35
9.6 Model L is much less grounded in the prompt . . . 36
9.7 Successes . . . 36

10 Phase 3 — Attention After Finetuning **37**
10.1 The method, and why it is fair . . . 37
10.2 Results . . . 37
10.2.1 Per-head shifts . . . 37

11 The Question Bank **39**
11.1 On the code . . . 39
11.2 On the data . . . 40
11.3 On architecture choices . . . 40
11.4 On training . . . 41
11.5 On the figures . . . 41
11.6 On Phase 3 . . . 42
11.7 If you genuinely do not know . . . 43

12 Mistakes to Own **44**

13 What I Would Do Next **46**

14 The Four Required Questions **47**
14.1 How did data scale and quality differ? . . . 47
14.2 How do LM and reasoning results compare across tiers? . . . 47
14.3 What tokenizer / corpus factors most affected the lower-resource model? . . . 47
14.4 What evidence explains the observed differences? . . . 48

15 Final Checklist Before Walking In **49**

Chapter 1

The Cheat Sheet

If you forget everything else, remember this chapter.

1.1 What I built, in three sentences

1. I built **two completely separate** GPT-style language models from scratch in PyTorch — one for **Hindi** (called **Model H**, the higher-resource language) and one for **Nepali** (called **Model L**, the lower-resource language). They share no data, no tokenizer, no vocabulary and no weights.
2. I collected and cleaned the corpus myself, trained a tokenizer for each language separately, wrote the whole Transformer using only basic PyTorch layers, pretrained both models for about 11 hours each on a Tesla T4 GPU, and evaluated them.
3. Then I generated my own synthetic reasoning dataset in each language, finetuned each model on its own language only, and measured whether that taught the model to reason. **It did not** — and the most interesting part of my project is the evidence showing why.

1.2 The three findings I lead with

1. **Perplexity and bits-per-byte disagree.** Perplexity says Hindi is 41% better. Bits-per-byte says Nepali is slightly better. Both numbers are correct — most of the gap is a *tokenizer artifact*, not a real quality difference.
2. **Both models independently grew the same attention structure.** Different languages, different tokenizers, different random seeds — and both produced a "previous-token head" at layer 3. Nobody designed that.
3. **Finetuning taught the output format, not the reasoning.** Exact match jumped from 0% to 35%, which looks like success. It is not: on a fair forced-choice test both models sit at or below the 41.67% chance rate.

1.3 Numbers to memorise

Architecture (identical for both)	
Layers / heads / model dim / FFN dim	6 / 8 / 512 / 2048
Head dimension d_k	64
Context length	512
Dropout	0.1, at four places

	Model H	**Model L**
Total parameters	23,273,472	24,297,472
Non-embedding parameters	18,915,328	18,915,328
Vocabulary	8,000 (Unigram)	10,000 (BPE)
Corpus tokens	505,852,028	635,204,333
Train-split tokens	485,956,678	521,426,359
Bytes per token	8.875	11.441
Test perplexity	**27.41**	46.68
Test bits-per-byte	0.5382	**0.4846**
Greedy repetition rate	0.598	0.577
Best chrF++ (at $T = 1.0$)	16.29	18.05
Reasoning exact match	**35.15%**	29.45%
Reasoning forced choice	36.35%	39.30%
Forced-choice chance rate	41.67%	41.67%

1.4 The one thing I must never get wrong

I did NOT implement RoPE

My positional encoding options are `"learned"`, `"sinusoidal"` and `"none"`. I shipped **learned absolute**.

If the examiner asks about RoPE, the correct answer is to explain what RoPE does and why I chose differently. Claiming I built it would collapse the moment they open `model/transformer.py`. See Section 4.3.

Chapter 2

What This Project Is, In Plain Words

2.1 The task

A language model is a program that, given some text, predicts the next piece of text. That is all it does. Everything else — answering questions, writing essays — comes out of doing that one thing very well.

The assignment asked me to build two of these, for two Indian languages with different amounts of available data, and then compare them fairly. The point is not to win a competition. The point is to understand *why* the two models differ.

In simple words: Higher-resource vs lower-resource language

A "higher-resource" language has a lot of text freely available on the internet — Hindi has hundreds of millions of words easily reachable. A "lower-resource" language has much less — Nepali has far fewer websites, fewer digitised books, fewer newspapers online.
The interesting research question is: *when you give both languages the same model size and roughly the same number of tokens, what actually differs?*

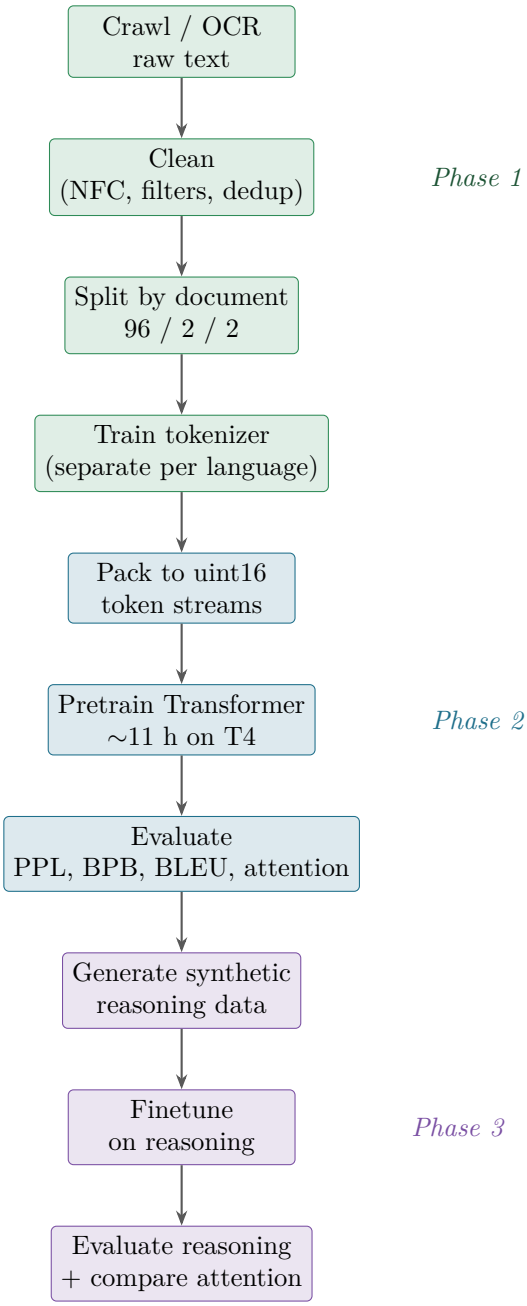
2.2 Why two separate models and not one multilingual model

This was a hard rule in the assignment, and there is a good scientific reason for it. If I trained one model on both languages with one shared vocabulary, then any difference I measured could be explained by the two languages sharing or competing for capacity. By keeping them completely separate, and by giving them *exactly the same architecture*, every difference in the results must come from the data or the tokenizer.

In my project: Identical non-embedding parameters

Both models have exactly **18,915,328** non-embedding parameters. Same depth, same width, same number of heads, same feed-forward size. The only difference in total parameter count comes from the vocabulary size, which changes the embedding table.
This is what lets me say "the difference is data and tokenizer, not capacity" and have it actually be true.

2.3 The shape of the whole pipeline



Everything in this diagram was done **twice**, once per language, with no sharing between the two columns.

Chapter 3

Phase 1 — Data and Tokenizers

3.1 Where the text came from

	Model H (Hindi)	Model L (Nepali)
Final documents	704,101	1,918,265
Corpus tokens	505,852,028	635,204,333
Manual share (token-weighted)	30.66%	26.41%

"Manual share" means the fraction of tokens I collected myself — by crawling sites and running OCR on scanned material — rather than downloading a ready-made dataset. The assignment required a real manual component, so this number matters.

In simple words: OCR — Optical Character Recognition

OCR is software that looks at a *picture* of text (a scanned book page, a photograph of a newspaper) and produces the actual characters. It is needed because a lot of Indian-language material exists only as scans, not as typed text.
OCR makes mistakes, especially with Devanagari conjunct characters, which is one reason the cleaning stage afterwards is so aggressive.

3.2 Cleaning — what I did and why, step by step

The examiner asked about this last time. Give it as a *pipeline*, in order.

3.2.1 Step 1 — Unicode normalisation (NFC)

In simple words: Why NFC matters for Devanagari

The same visible Devanagari string can be stored in the computer in more than one way. For example a character with a mark on it can be stored as one combined code point, or as a base character followed by a separate mark. They look identical on screen but have different bytes. If I do not normalise first, then:

- deduplication misses real duplicates, because two identical documents have different bytes;

- the tokenizer learns two different pieces for what is really one letter, wasting vocabulary slots.

NFC is the "composed" normal form — it puts everything into the combined representation. I run it *first*, before anything else.

I also strip zero-width characters and control characters at this stage. These are invisible characters that often come from web scraping and that would otherwise survive into the training data.

3.2.2 Step 2 — Strip pure-Latin tokens

Hindi web pages contain a lot of English — navigation menus, advertisements, "Click here". I removed 8,016,102 pure-Latin tokens from the Hindi corpus. The goal is a *monolingual* model, so English text is noise.

3.2.3 Step 3 — Drop bad documents

Here is the actual funnel for Hindi:

Stage	**Documents dropped**
Malformed Unicode	17
Too short (under 200 characters)	63,132
Low Devanagari ratio	228
Detected as Bhojpuri (reject filter)	1,365
Language-ID mismatch	1,778
Exact or near-duplicate	2,983
Documents in	**773,606**
Documents kept	**704,103**
Characters in	**1,840,356,619**
Characters kept	**1,764,598,107**
Retention	**95.9%**

In simple words: Why drop very short documents?

A 50-character document is usually a navigation fragment, an error page, or a caption. It carries almost no language structure for the model to learn from, but it still costs a slot in the corpus and can bias the statistics. The 200 character threshold is a judgement call, documented in the config.

In simple words: Why remove duplicates?

If the same news article appears 50 times in the corpus, the model sees it 50 times and partly memorises it. Worse, if one copy lands in the training set and another in the test set, the test score becomes a measure of memorisation rather than of language modelling. That is called **leakage**.

The Nepali pipeline is organised differently — as numbered stages `S00` to `S14` — and adds two things Hindi does not have:

- a **perplexity filter** (stage S06, kept 97.02%): it scores each document with a simple model and drops text that scores very badly, which catches garbled OCR and machine-generated spam;
- **MinHash / LSH** near-duplicate removal (stage S08, kept 97.61%).

In simple words: MinHash / LSH in one paragraph

Comparing every document to every other document is impossible when there are two million of them — that is two trillion comparisons. MinHash creates a short "fingerprint" of each document such that similar documents get similar fingerprints. LSH (Locality-Sensitive Hashing) then puts documents with similar fingerprints into the same bucket, so only documents inside a bucket need real comparison. It turns an impossible problem into a fast one.

3.3 Splitting into train, validation and test

Split	Hindi	Nepali
Train	675,937	1,520,000
Validation	14,082	199,428
Test	14,082	198,837

Hindi is about **96 / 2 / 2** by document count, shuffled with a fixed seed (42) so it is reproducible.

The real answer to "how did you split?"

The important word is **document level**.
If I had split *after* chopping text into chunks, then two chunks of the same newspaper article could end up one in training and one in test. The model would have seen almost the same sentences already, and the test perplexity would be flattering nonsense.
Splitting whole documents means a test document was never seen in any form. `validate_phase1.py` checks this and reports zero overlap.

3.4 Tokenizers

In simple words: What a tokenizer is

A neural network cannot read letters. It needs numbers. A tokenizer is the thing that cuts text into pieces ("tokens") and gives each piece a number.
The pieces are usually not whole words and not single letters — they are somewhere in between, called subwords. Common words become one token; rare words get split into several pieces. This way a small vocabulary can still represent any text.

	Model H (Hindi)	**Model L (Nepali)**
Algorithm	SentencePiece Unigram	SentencePiece BPE
Vocabulary size	8,000	10,000
Sizes swept	5,000–10,000	swept, 10,000 chosen
Bytes per token	8.875	11.441
Unknown rate on test	0.000000	0.001894

In simple words: BPE vs Unigram, in simple words

BPE (Byte Pair Encoding) starts from single characters and repeatedly glues together the most frequent adjacent pair. It builds up, greedily.
Unigram works the other way: it starts with a large set of candidate pieces and repeatedly *throws away* the pieces that help least, keeping the set that best explains the text probabilistically.
Unigram usually gives slightly cleaner splits for morphologically rich languages; BPE is simpler and faster. I swept both languages and chose by measurement rather than by preference.

In simple words: Fertility

Fertility means "how many tokens does one word turn into, on average". Lower is better — it means the vocabulary fits the language well. If fertility is high, every sentence becomes a long sequence, which wastes context length and makes prediction harder.

3.4.1 Why the vocabulary sizes differ

Every extra 1,000 vocabulary entries costs $1{,}000 \times 512 = 512{,}000$ parameters out of a roughly 25M budget. So vocabulary size is a real trade-off: a bigger vocabulary means shorter sequences and lower fertility, but fewer parameters left for actual computation.

I chose each size from *that language's own measurements* — fertility and unknown-token rate on held-out text, traded against the embedding budget. They landed on different numbers because the languages are different. Vocabularies are never shared; that is a hard constraint of the assignment.

A discrepancy I should own

Phase 1 reported the Nepali test unknown-token rate as **0.0**. The measured value is actually **0.0019**.
The cause is that the Nepali BPE was trained *without byte fallback*, so characters it has never seen become `<unk>` instead of being broken into bytes. Mostly this is Latin characters inside mixed text.
I found something much sharper in Phase 3 — see Section 7.4.2. If the examiner spots this, agree immediately and explain the cause.

Chapter 4

Phase 2 — The Transformer, Piece by Piece

Every part of this chapter is code I wrote myself using only `nn.Linear`, `nn.Embedding`, `nn.Dropout` and plain tensor operations. No `nn.Transformer`, no `nn.MultiheadAttention`, no `F.scaled_dot_product_attention`, no HuggingFace. There is a test that walks the module tree and fails if any of those appear.

4.1 The forward pass in tensor shapes

If the examiner says "walk me through the forward pass", answer in **shapes**, not adjectives.

```
idx (B, T) token ids
token_embedding (B, T, 512) nn.Embedding(vocab, 512)
+ position (T, 512) broadcasts over the batch
embedding_dropout p = 0.1

per block (x6), pre-norm:
  LayerNorm -> W_q / W_k / W_v (B, T, 512) each
  reshape (B, 8, T, 64) 64 = 512/8 = d_k
  Q @ K^T / sqrt(d_k) (B, 8, T, T)
  + causal mask (-inf above diagonal)
  softmax -> attn_dropout -> @ V (B, 8, T, 64)
  merge heads (B, T, 512)
  W_o -> resid_dropout
  + residual
  LayerNorm -> 512 -> 2048 -> GELU -> 512 -> dropout
  + residual

final LayerNorm
lm_head (B, T, vocab) tied to token_embedding
```

4.2 Step 1 — Turning tokens into vectors

In simple words: Embedding

An embedding is just a big lookup table. Token number 4,213 means "go to row 4,213 of a table and take those 512 numbers". Those 512 numbers are the model's representation of that token, and they are learned during training.
So the embedding table has shape (vocabulary size × 512). For Hindi that is 8,000 × 512 = 4,096,000 numbers.

4.3 Step 2 — Telling the model about position

In simple words: Why position has to be added separately

Self-attention treats its input as a *set*, not a sequence. If you shuffle the tokens, the outputs shuffle in exactly the same way and nothing else changes. The technical word is **permutation-equivariant**.
That means without position information the model literally cannot tell राम श्याम से लंबा है from श्याम राम से लंबा है — "Ram is taller than Shyam" versus "Shyam is taller than Ram". Opposite meanings, same bag of tokens.

4.3.1 What I used

Learned absolute positional embeddings. A second lookup table, one row per position: 512 × 512 = 262,144 parameters. Row 0 is added to whatever token sits at position 0, row 1 to position 1, and so on.

In my project: Why learned and not sinusoidal

Sinusoidal encodings are a fixed mathematical formula rather than learned numbers. Their main advantage is that they are defined for *any* position, so a model can in principle handle sequences longer than it trained on.
I train and evaluate at one fixed length — 512 — so that advantage never pays off. Learned positions let the model spend positional resolution where its own data needs it.

The cost, which I state myself: a hard ceiling. There is no embedding row for position 512, so the model cannot run on longer sequences without retraining that table. `forward()` raises an error if $T >$ `block_size`.

4.3.2 RoPE — what it is and why I did not use it

Be completely honest here

I did **not** implement RoPE. My options are `"learned"`, `"sinusoidal"` and `"none"`.

In simple words: RoPE (Rotary Position Embedding)

RoPE does not add anything to the embeddings at all. Instead it *rotates* the query and key vectors *inside* the attention function, by an angle that depends on the position.
The trick is this: when you take the dot product of two rotated vectors, the result depends only on the *difference* between their angles. So the attention score between position i and position j becomes a function of $i - j$ — it is **relative**, not absolute.

	Learned absolute (mine)	**RoPE**
Where it is applied	embedding layer, once	inside attention, every layer
What it encodes	absolute index	relative offset $i - j$
Extra parameters	262,144	0
Beyond trained length	impossible	degrades gracefully

Say this out loud

"RoPE is the better default for a model meant to generalise beyond its context window, and it costs zero parameters. But I train and evaluate at a single fixed length of 512, so the property it buys is one I never exercise. It would also mean changing the attention function itself rather than the embedding layer, and the brief says explicitly that this is not an architecture bake-off — so I spent the complexity budget elsewhere."

4.4 Step 3 — Multi-head causal self-attention

This is the heart of the model and the most likely place for detailed questions.

In simple words: Attention in one paragraph

For every position in the sentence, attention asks: "which other positions should I look at, and how much?" It answers by comparing a **query** vector for the current position against a **key** vector for every other position. Where query and key match well, the score is high. Those scores are turned into percentages with softmax, and then used to take a weighted average of the **value** vectors.
So: query = what I am looking for, key = what I advertise, value = what I actually give you.

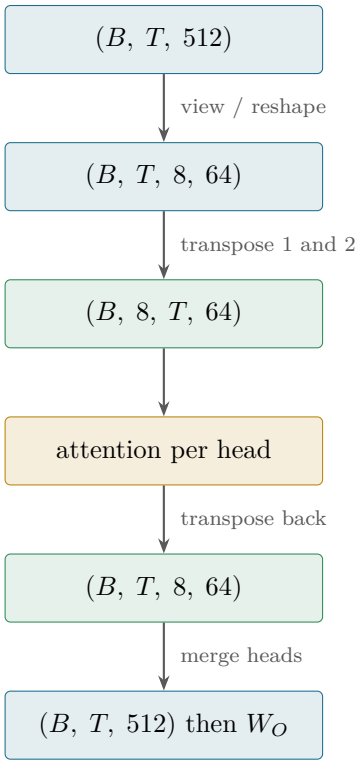
4.4.1 The three projections

$Q = XW_Q$, $K = XW_K$, $V = XW_V$. Three separate `nn.Linear` layers, each mapping 512 numbers to 512 numbers. Then a fourth, W_O, at the end.

4.4.2 Splitting into heads — the reshape

In simple words: Why multiple heads

One attention pattern per layer would be very restrictive — the model could only look at one kind of relationship at a time. Instead we split the 512 dimensions into 8 groups of 64. Each group is a "head" and runs attention independently, so one head can track grammatical agreement while another tracks the previous token and another tracks the subject of the sentence.



Say this out loud

"The reshape is $(B, T, 512) \to (B, T, 8, 64) \to (B, 8, T, 64)$, so each head attends inside its own 64-dimensional subspace, then the exact inverse to concatenate the heads back before the output projection W_O."

4.4.3 Why divide by $\sqrt{d_k}$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

In simple words: The scaling, explained simply

Q and K entries are roughly zero-mean with variance about 1. A dot product adds up $d_k = 64$ such products, so its variance is about 64 and its standard deviation about 8.
Feeding numbers of size ± 8 into softmax makes it almost one-hot: one value gets essentially all the probability. And when softmax is saturated like that, its gradient is almost zero, so

learning stops.
Dividing by $\sqrt{d_k} = 8$ brings the scores back to roughly unit variance, which keeps softmax in the range where it responds to changes.

4.4.4 The causal mask

In simple words: Why the mask exists

The model is trained to predict the next token. If position 5 could see position 6, predicting position 6 would be trivial — just copy. The model would score perfectly during training and be useless at generation time, when the future does not exist yet.
So we block it: position t may attend only to positions $\leq t$.

The mask is an upper-triangular matrix of $-\infty$, added to the scores *before* softmax. It is stored as a buffer and sliced to `[:T,:T]`.

In my project: Proving the mask works — exactly

The assignment asks to verify this empirically. My test (`test_causal_mask_blocks_the_future`) changes every token after position t and measures how much the logits at positions $\leq t$ move.
The answer is **exactly 0.000e+00**. Not "within tolerance".

Say this out loud

"It is exactly zero, not approximately zero, because softmax$(-\infty)$ is exactly 0 and $0 \times v$ is exactly 0 in floating point. The future contributes literally nothing, not merely something small."

4.5 Step 4 — The rest of the block

4.5.1 Feed-forward network

Two linear layers with GELU in between: $512 \to 2048 \to 512$.

In simple words: What the feed-forward layer is for

Attention moves information *between* positions. The feed-forward network processes each position *on its own*, with no mixing. It is where most of the model's parameters live and where a lot of the actual "knowledge" is stored. The wide middle layer (4×) gives it room to compute something non-linear before compressing back.

4.5.2 Residual connections

In simple words: Residual connections

Instead of $x \to f(x)$, we do $x \to x + f(x)$. Each sublayer *adds* a correction rather than replacing everything.
This matters for training: during backpropagation the gradient has a direct path straight through the additions back to layer 0, so it does not have to survive being multiplied by six layers of weights. Without residuals, deep stacks simply do not train.

4.5.3 Pre-norm vs post-norm

I use **pre-norm**: LayerNorm is applied *before* each sublayer, not after.

In simple words: Why pre-norm is more stable

With post-norm, the residual path passes through a LayerNorm at every single block, so the gradient gets rescaled six times on its way back. With pre-norm the residual path is clean — an unbroken sum from the output all the way to the embedding.
Post-norm can be trained, but it needs careful learning-rate warmup and is fragile in deep stacks. Pre-norm is what almost all modern models use.

4.5.4 LayerNorm

In simple words: LayerNorm in simple words

For each position separately, take its 512 numbers, subtract their mean and divide by their standard deviation, then scale and shift by learned parameters.
The effect is to keep the numbers flowing through the network in a consistent range, so that no layer suddenly receives inputs that are a thousand times bigger than it expects.

4.6 Step 5 — Output and weight tying

The final LayerNorm is followed by a linear layer mapping 512 to the vocabulary size, giving one score per possible next token.

In my project: Weight tying

I reuse the *input embedding matrix* as the output projection, transposed. This saves $8{,}000 \times 512 = 4{,}096{,}000$ parameters — about 15% of **Model H**.
The justification is that both matrices represent the same thing: a mapping between tokens and vectors. There is no reason to learn that mapping twice.

4.7 The training objective

Cross-entropy between the logits at position t and the actual token at position $t+1$, averaged over all positions.

In simple words: Cross-entropy, simply

The model outputs a probability for every possible next token. Cross-entropy is $-\log$(probability it gave to the correct token).
If it gave the right answer probability 1, the loss is 0. If it gave it probability 0.01, the loss is about 4.6. So the loss punishes confident wrong answers very heavily.

In my project: How the shift is done

In my data loader, `x = stream[o : o+block_size]` and `y = stream[o+1 : o+1+block_size]`. The targets are simply the inputs shifted by one position. The model never sees its own target because of the causal mask.

4.8 Dropout — all four places

The examiner asked about this. Name all four sites, all at $p = 0.1$:

1. **Embedding dropout** — on the summed token + position embedding, before the first block.
2. **Attention dropout** — on the softmax *probabilities*, after softmax and before multiplying by V.
3. **Residual dropout** — on the attention output after W_O, before the residual addition.
4. **Feed-forward dropout** — on the FFN output.

In simple words: What dropout does

During training, dropout randomly sets a fraction of the numbers to zero. This stops the model from relying too heavily on any one feature, because that feature might not be there next time. It is a way of forcing redundancy, which reduces overfitting.

In my project: Site 2 is the interesting one

Dropping *attention probabilities* is different from ordinary dropout. It randomly cuts attention edges, so a head cannot stake everything on a single key position. It regularises the *routing* of information, not just the features.
A detail worth knowing: after dropout the attention rows no longer sum to 1. That is accepted. PyTorch uses "inverted" dropout, which scales the surviving values by $1/(1-p)$ during training, so the *expected* value is preserved.

Is dropout on during evaluation?

No. `model.eval()` disables every `nn.Dropout`, and I call it before validation, before `generate()`, and inside the attention tooling.
Because PyTorch uses inverted dropout, nothing needs rescaling at test time.

Likely follow-up: "what if you left it on?" Perplexity would be inflated *and* non-deterministic — a different random mask every pass, so the same test set would give a different number each time. Every metric in my report depends on `eval()` having been called.

Chapter 5

Phase 2 — Training

5.1 The settings, and why

Setting	Value and reason
Optimiser	AdamW, $\beta = (0.9, 0.95)$
Weight decay	0.1, *on matrices only*
Peak learning rate	6×10^{-4}
Minimum learning rate	6×10^{-5}
Warmup	500 steps, then cosine decay
Effective batch	$32 \times 2 \times 512 = 32{,}768$ tokens per step
Precision	fp16 with GradScaler (T4 has no bf16)
Gradient clipping	1.0

In simple words: AdamW vs Adam

Both adapt the learning rate per parameter. The difference is how weight decay is applied.
In plain Adam, the L2 penalty is added into the gradient, and then gets divided by the second-moment estimate along with everything else — so parameters with small gradients get *more* decay. That is not what anybody wants.
AdamW "decouples" it: the decay is applied directly to the weights, separately from the gradient step. That is the correct behaviour.

In my project: Decay on matrices only

I apply weight decay only to 2-D parameters (the weight matrices). Biases, LayerNorm gains and embeddings are excluded.
The reason: those parameters set *scales and offsets*. They do not perform the linear mixing that weight decay is meant to regularise, and shrinking them toward zero hurts more than it helps.

In simple words: Warmup and cosine decay

At the very start, the model's weights are random and the Adam moment estimates are meaningless. Taking a big step then can throw the model somewhere it never recovers from. So

warmup raises the learning rate linearly from almost zero over the first 500 steps.
After that, cosine decay slowly lowers it. Large steps early explore; small steps later settle into a good minimum.

In simple words: Gradient accumulation

`batch_size` is limited by how much fits in GPU memory. `grad_accum_steps` lets me run several micro-batches and add up their gradients before taking one optimiser step.
So the *effective* batch is 32 sequences × 2 accumulation steps × 512 tokens = 32,768 tokens per update. Both models use exactly the same number, so their token budgets match.

In simple words: fp16 and the GradScaler

fp16 (16-bit floats) is faster and uses less memory, but it has a narrow exponent range. Small gradients can underflow to zero and be lost.
The GradScaler multiplies the loss by a large factor before backpropagation, so the gradients land in a representable range, then divides it out before the optimiser step. bfloat16 does not need this because it has a wider exponent range — but the Tesla T4 (compute capability 7.5) does not support bfloat16.

5.2 What actually happened

	Model H	**Model L**
Steps completed	14,520	14,440
Tokens seen	475,791,360	473,169,920
Wall clock on Tesla T4	11.0 h	11.0 h
Throughput	∼12.0k tokens/s	∼12.0k tokens/s
Best checkpoint	step 14,500	step 14,000

5.3 Checkpoints and resume

In my project: What is inside a checkpoint

- model weights
- optimizer state (Adam's two moment estimates per parameter)
- scheduler state
- current step, tokens consumed, best validation loss
- the full run configuration
- **both RNG states** — torch and numpy

Resume was verified across three consecutive sessions before the real runs began.

In simple words: Why the RNG states matter

Most people forget these. Without them, a resumed run draws a different order of training data and different dropout masks. The run still works, but it is no longer the *same experiment* — and reproducibility is an explicit requirement of the assignment.

A real bug I found and fixed

My learning-rate schedule originally restored its *horizon* (`max_steps`) from the checkpoint. When I resumed with a *larger* `max_steps` — the normal case when a session dies and you restart with a longer run — the restored horizon left `step > max_steps`, which pinned the learning rate at its floor for the whole extension. Training looked stalled and I nearly misdiagnosed it as a data problem.
The fix: restore *progress* only. The schedule's shape comes from the live config.

Chapter 6

Phase 2 — Evaluation and What the Numbers Mean

6.1 Perplexity and bits-per-byte — the headline

Test metric	Model H	Model L	Winner
Cross-entropy (nats)	3.3109	3.8433	H
Perplexity	27.41	46.68	H, by 41%
Bits-per-byte	0.5382	0.4846	L
Bytes per token	8.875	11.441	—

Both scored over 1,999,872 tokens per split.

In simple words: Perplexity

Perplexity is just $e^{\text{cross-entropy}}$. It has a nice interpretation: it is roughly "how many options is the model effectively choosing between at each step".
A perplexity of 27 means the model is about as uncertain as if it were picking uniformly among 27 equally likely tokens.

Why perplexity is unfair across different tokenizers

Perplexity is measured **per token**. But a token is not a fixed amount of text!
Nepali's tokens carry 11.44 bytes of text on average; Hindi's carry 8.88. So a Nepali token is a *bigger prediction* — more text has to be guessed at once. Of course it is harder.
Comparing perplexity across two different tokenizers is like comparing "errors per page" between a book with 200 words per page and one with 500.

In simple words: Bits-per-byte fixes this

Bits-per-byte divides the total information cost by the number of *UTF-8 bytes* of the original text, not by the number of tokens. Bytes are the same unit for both languages, so the tokenizer disappears from the denominator.

Be able to derive it live. This is the most impressive twenty seconds available to you in the room:

$$\text{BPB} = \frac{\text{CE}_{\text{nats}}}{\ln 2 \times \text{bytes per token}}$$

$$\text{Model H:} \quad \frac{3.3109}{0.6931 \times 8.875} = 0.5382 \qquad \text{Model L:} \quad \frac{3.8433}{0.6931 \times 11.441} = 0.4846$$

Say this out loud

"Both numbers are correct; they measure different things. Nepali's BPE packs 11.44 bytes into a token against Hindi's 8.88, so its tokens are individually harder to predict but each one carries more text. Normalising by UTF-8 bytes takes the tokenizer out of the denominator — and then the gap almost vanishes and actually reverses.
Reporting perplexity alone would have supported the conclusion that the low-resource model is much worse. It is not."

6.2 Generation quality and the temperature question

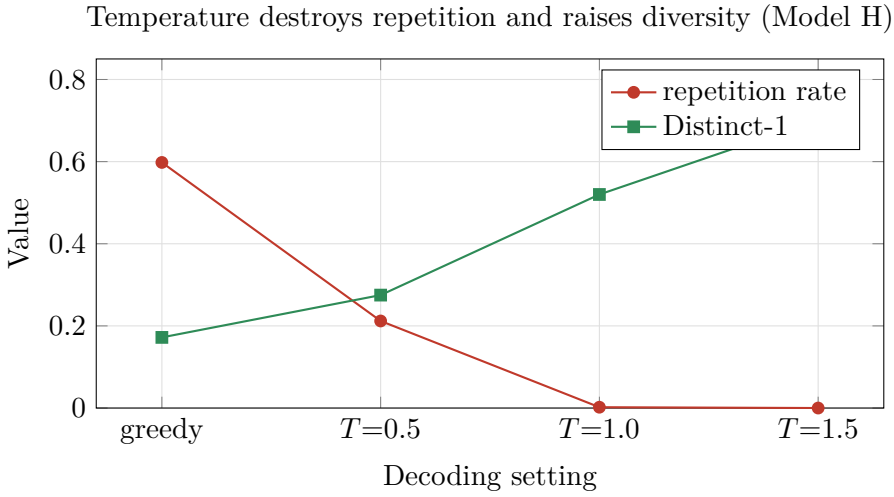
In simple words: Temperature

When the model generates text, it produces a score for every possible next token. Temperature divides those scores before turning them into probabilities:

$$P(\text{token}) = \text{softmax}(\text{logits}/T)$$

$T \to 0$ means always take the highest score (greedy). Large T makes everything nearly equally likely (random).

Model H setting	**Repetition**	**Distinct-1**	**BLEU-4**	**chrF++**
greedy	0.598	0.172	1.96	12.49
$T = 0.5$	0.212	0.275	1.80	14.33
$T = 1.0$	**0.002**	0.520	1.01	**16.29**
$T = 1.5$	0.000	0.702	0.16	14.67
Model L setting				
greedy	0.577	0.179	2.37	14.48
$T = 0.5$	0.190	0.345	1.88	16.54
$T = 1.0$	**0.001**	0.634	1.69	**18.05**
$T = 1.5$	0.000	0.793	0.78	16.13



In simple words: Why low temperature causes repetition loops

At low temperature the highest-scoring token always wins. So if the model ever returns to a state it has been in before, it produces exactly the same continuation — and then returns to that state again. It is a deterministic loop with nothing random to break it.
Raising the temperature introduces enough randomness to escape.

Say this out loud

"Greedy decoding *degenerates* — 59.8% of generated 4-grams are repeats and the samples visibly loop. Raising temperature monotonically destroys that repetition, down to 0.2% at $T = 1.0$. Model L behaves identically: 0.577 down to 0.001."

6.2.1 The subtle part — BLEU and chrF++ disagree

- **BLEU falls monotonically** (1.96 → 0.16). BLEU counts exact n-gram matches against one reference. Greedy's repeated common n-grams accidentally match, so BLEU actually *rewards* the degenerate output.
- **chrF++ peaks at** $T = 1.0$ (16.29 for H, 18.05 for L). chrF++ scores *character* n-grams, so it gives credit for correct Devanagari morphology even when the exact words differ from the reference.

Say this out loud

"$T = 1.0$ removes the repetition *and* peaks chrF++, so diversity costs no fidelity up to that point. It also shows why BLEU is a poor metric for open-ended generation in a morphologically rich language — a degenerate loop can outscore a fluent continuation."

In my project: Why I wrote ROUGE-L myself

The standard `rouge_score` package strips non-ASCII characters, which zeroes out Devanagari completely — every score would have been 0.0.
Mine is a longest-common-subsequence over whitespace tokens.

6.3 Attention analysis

Phase 2, $T = 512$, uniform baseline 5.2565 nats	**Model H**	**Model L**
Mean attention entropy	3.205	3.090
Mean attention distance	62.36	54.51
Previous-token head (layer 3)	0.06 nats @ 1.3	0.14 nats @ 1.1

In simple words: Attention entropy

Entropy measures how spread out a probability distribution is. Low entropy means the head is focused on one or two positions. High entropy means it is spread thinly over everything. The "uniform baseline" is what the entropy would be if a head attended equally to everything it is allowed to see. Being far below the baseline proves the model learned structure rather than just averaging.

If they ask where 5.2565 comes from

It is **not** $\ln(512) = 6.24$.
Under causal masking, query position t can only see $t + 1$ keys. So the entropy of uniform attention at that position is $\ln(t + 1)$, which is small at the start of the sequence and large at the end.
The baseline is the **mean of** $\ln(t+1)$ **over all positions**, which for $T = 512$ comes to 5.2565.

In simple words: Mean attention distance

For each query position, how far back does it look on average? A head with distance 1.3 is looking at the immediately preceding token. A head with distance 62 is looking far back across the sequence.

Say this out loud

"Both models — different languages, different tokenizers, different seeds — **independently converged on a previous-token head at layer 3**: entropy 0.06 at distance 1.3 for H, 0.14 at 1.1 for L. Nobody designed that. And mean entropy sits about 2 nats below the uniform baseline, so attention is genuinely structured, not diffuse."

6.3.1 Reading a heatmap out loud

Point at three things, in this order:

1. **The upper triangle is uniformly zero.** That is the causal mask, visually — the same fact the test proves numerically.
2. **The bright first column** — an "attention sink" at position 0. Softmax must sum to 1, so when a head has nothing specific to attend to it parks probability mass on the first token.
3. **The tight diagonal band** in early layers means local, positional heads. Late layers show off-diagonal structure that follows content.

Chapter 7

Phase 3 — The Reasoning Dataset

7.1 What the assignment asked for

Build *my own* synthetic dataset of comparison and transitive-reasoning questions, in each model's own language, generated programmatically so that every correct answer is known by construction. Do not download an existing benchmark.

7.2 The four task families

Family	Hops	Candidates	Example (Hindi)
`compare2`	1	2	वजिय सोहन से धीमा है। दोनों में से तेज़ कौन है?
`transitive3`	2	3	मीना प्रीत से छोटी है। प्रीत सुनीता से छोटी है।
`numeric2`	1	2	संगीता की उम्र १३ साल है। रेखा की उम्र ६८ साल है।
`numeric3`	1	3	three measured values, pick the extreme

Five comparable attributes: height, age, weight, speed and wealth. The last two have no natural numeric phrasing in either language, so they appear only in the comparison tasks.

In my project: Direction variation

Both the *stated* direction and the *asked* direction vary independently. If the sentence says "A is taller than B" and the question asks who is *shorter*, the answer is B.
Without this the answer would simply always be the first-named entity. Measured: the answer is the first-named entity in 46% of examples — i.e. chance.

7.3 Size and leakage control

Split	Examples	Templates	Entity names
train	20,000	14	40
validation	2,000	4	10
test	2,000	4	10

Identical for both languages, and the two corpora share **no sentence, no entity name and no template** — verified programmatically.

In simple words: Train–test leakage in a synthetic dataset

If I generate 20,000 training examples and 2,000 test examples from the same pool of names and the same sentence templates, the model can score well by memorising "this name goes with this answer" or "this phrasing means that". The test score would then measure memorisation, not reasoning.

In my project: Both defences, applied together

- **Held-out entity names** — 40 / 10 / 10, disjoint. A test question is always about people the model never saw in training.
- **Held-out templates** — 14 / 4 / 4, disjoint. A test question is always phrased in a way the model never saw either.

The generator **asserts** this rather than assuming it: if the pools ever overlap, it crashes instead of quietly inflating the test score.

This paid off directly — see Section 9.6.

7.4 Two defects I caught in my own dataset

Volunteer these. They show you reviewed your own work adversarially.

7.4.1 A 45% gender-agreement leak

The bug

Hindi adjectives inflect for gender: छोटा for a man, छोटी for a woman.
So a question ending सबसे छोटी कौन है? asked about one woman and two men has exactly *one grammatically possible answer* — the woman. A model could score well by matching grammatical agreement and never perform the comparison at all.
181 out of 400 examples — 45% — were solvable this way.

The fix: draw every entity in an example from the *same* gender. Then the adjective carries no information about which entity is meant, and the answer can only come from the comparison. Verified: zero mixed-gender examples.

7.4.2 Nepali numerals were completely unrepresentable

The bug

Model L's BPE vocabulary contains **no Devanagari digit at all** — not one of ०१२३४५६७८९ is a vocabulary piece — and it was trained *without byte fallback*.
So every Devanagari numeral encoded to `<unk>`. My first draft ran a **4.8% unknown-token rate** across the whole finetuning corpus, which would have trained Model L on holes exactly where the quantities belong.

The fix: the Nepali generator writes numerals with Arabic digits (`141`, `175`), which the vocabulary covers completely and which modern Nepali prose overwhelmingly uses anyway. Model H's Unigram vocabulary covers *both* digit systems at zero unknowns, so Hindi keeps Devanagari digits.

Say this out loud

"The asymmetry between the two generators is not an inconsistency. It is the low-resource tokenizer's coverage gap showing up in practice, and I report it as a finding rather than papering over it. Both corpora now tokenize at exactly 0.000000 unknown rate."

7.5 Natural phrasing in Nepali

The assignment requires the target language's script *and* natural phrasing. In Nepali the comparative postposition and the genitive attach to the preceding word:

रबनिको उचाइ 141 सेन्टमिटिर छ। and कल्पनाभन्दा

Writing them detached (रबनि को, कल्पना भन्दा) is the usual signature of Nepali that has been translated from Hindi. My templates attach them.

Chapter 8

Phase 3 — Finetuning

8.1 The protocol

In my project: Every rule of the specified protocol

- Each model starts from **its own** Phase 2 checkpoint — H from step 14,500, L from step 14,000. Never the other model, never from scratch.
- Each keeps **its own** Phase 1 tokenizer, so the embedding and output matrices keep their pretrained rows. The finetuner **asserts** that the tokenizer's vocabulary size matches the checkpoint rather than trusting the config file.
- Checkpoints use the same resume-capable format as pretraining, which is what lets the Phase 2 evaluation and attention tools load them unchanged.

8.2 Hyperparameters — identical for both models

Setting	Value	Why
Peak learning rate	6×10^{-5}	one tenth of the pretraining peak
Minimum LR	6×10^{-6}	cosine floor
Warmup	100 steps	
Batch size	32 examples	
Epochs	2 (1,250 steps)	
Optimiser	AdamW, $\beta = (0.9, 0.95)$, wd 0.1	matrices only
Gradient clip	1.0	
Precision	fp16 + GradScaler	T4 has no bf16
Evaluation	every 20 steps	
Loss	**answer tokens only**	

In simple words: Why a much smaller learning rate for finetuning

The model already contains a language model worth keeping. A pretraining-sized step would overwrite that knowledge before the new task is learned.
The classic failure mode is a model that answers in the right *format* but has forgotten how to

read the question. One tenth of the pretraining peak is a standard, conservative choice.

8.3 Two design decisions worth explaining

8.3.1 Loss on the answer only

In my project: How it works

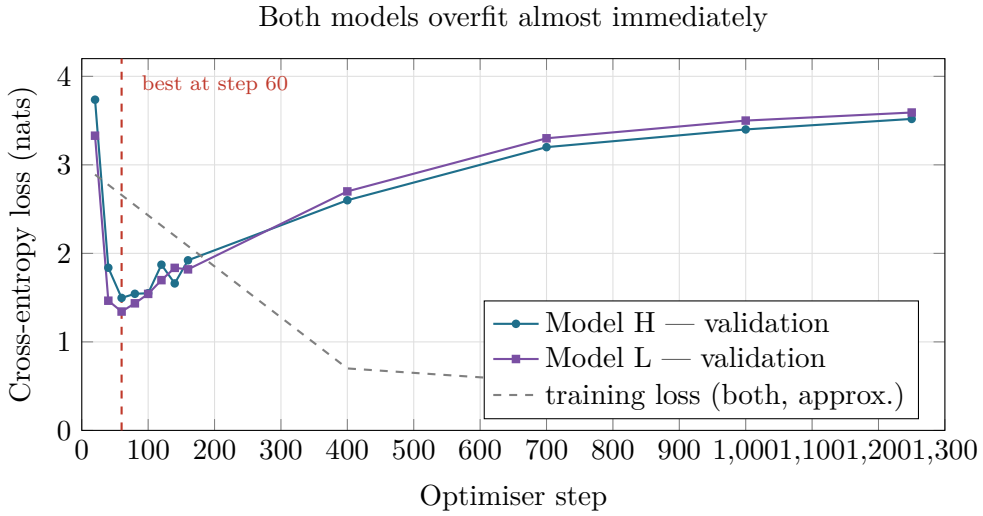
Prompt target positions are set to -1, and the model's `cross_entropy(..., ignore_index=-1)` already skips those.
If I trained on the prompt too, most of the gradient would go into reproducing question templates — which the model is *given for free* at test time — and the thing actually being measured (can it name the right entity) would get only a small share of the signal.

8.3.2 Bucketing by prompt length instead of padding

In simple words: Why padding would break things here

My models use *learned absolute* positional embeddings, so a token's representation depends on its index in the sequence.
If I pad on the *left*, every real token shifts to a position it never occupied during pretraining — the model sees them in the wrong place.
If I pad on the *right*, training is fine, but generation breaks, because generation appends new tokens at the end and the padding is in the way.
The solution: group examples whose prompts tokenize to exactly the same length. Then every sequence in a batch is genuinely aligned and no padding is needed in the prompt at all.

8.4 What happened during training



	Model H	**Model L**
Best validation loss	1.4966	1.3427
Step of best	**60**	**60**
Validation loss at step 1,250	3.5188	3.5911
Training loss at step 1,250	0.44	0.60

In simple words: What overfitting looks like, and why it happened here

Overfitting is when the model keeps getting better on the data it is trained on while getting *worse* on data it has not seen. The two curves separate.
Here it happened extremely fast — by step 60. The reason is the dataset design: with only 40 training names and 14 templates, memorising "the answer is usually one of these 40 names" is much easier than learning to read the question and copy the right name out of it.
Saving on *best validation loss* is what kept the evaluated checkpoints usable. Otherwise I would have reported the badly overfit final model.

A methodology mistake I found and fixed

My first run evaluated every **100** steps, and its best validation loss landed at step 100 — which is also exactly where warmup ends and the learning rate first reaches its peak. The checkpoint was being selected from a grid too coarse to find the real optimum.
Re-running with a **20**-step grid, changing nothing else, moved Model L from **14.90% to 29.45%** exact match and Model H from 32.70% to 35.15%.
The conclusions did not change — but the numbers did, and the coarse grid would have understated both models.

Chapter 9

Phase 3 — Results and What They Really Mean

9.1 Why I report two metrics

The problem with exact match alone

A *pretrained* model has never seen this prompt format. It will not spontaneously answer with a bare name. So its exact match is 0% — and that 0% is a **floor artefact**, not a measurement of reasoning ability.
Comparing "0% before" to "35% after" therefore tells you the model learned the output format. It tells you nothing about whether it learned to compare.

In simple words: Forced choice — the fair test

Instead of letting the model generate freely, I give it the entities the question actually names as candidates. For each candidate I append it to the prompt and compute the model's mean log-probability per answer token. The highest-scoring candidate is taken as the model's answer. Now chance is *known*: with 2 candidates it is 50%, with 3 it is 33.3%. Averaged over my test set it is **41.67%**. So the pretrained baseline becomes interpretable rather than a floor.

In my project: Why length-normalised

Names tokenize into different numbers of pieces. Without dividing by the token count, a one-token name beats a three-token name on *total* log-probability, for reasons that have nothing to do with the question.

9.2 The headline table

Model	Metric	Pretrained	Finetuned	Change
H	Exact match	0.00%	**35.15%**	+35.15
H	Forced choice	39.80%	36.35%	−3.45
H	Answer-token loss	5.2439	**0.9408**	−4.3031
L	Exact match	0.00%	**29.45%**	+29.45
L	Forced choice	38.80%	39.30%	+0.50
L	Answer-token loss	5.5367	**1.4351**	−4.1016
forced-choice chance rate		*41.67%*	*41.67%*	—

Say this out loud

"Exact match went from 0% to 35%, which looks like success. It is not. On a forced choice between the entities the question actually names, both models are **at or below the 41.67% chance rate**. Finetuning taught the output format, not the comparison."

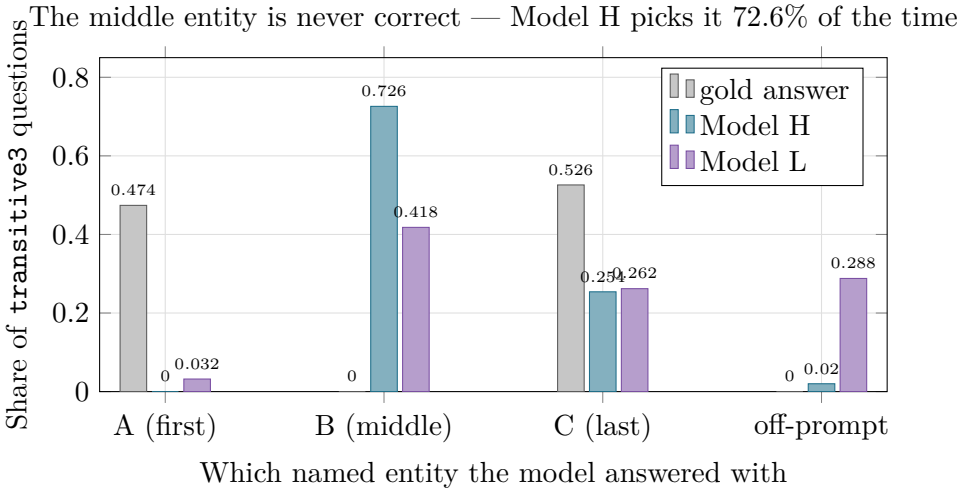
9.3 Per task family — the agreement with chance

Model	Family	Cands	Chance	Pretrained	Finetuned
H	`compare2`	2	50.0%	0.00%	47.40%
H	`numeric2`	2	50.0%	0.00%	47.40%
H	`numeric3`	3	33.3%	0.00%	33.40%
H	`transitive3`	3	33.3%	0.00%	**12.40%**
L	`compare2`	2	50.0%	0.00%	40.20%
L	`numeric2`	2	50.0%	0.00%	38.00%
L	`numeric3`	3	33.3%	0.00%	26.20%
L	`transitive3`	3	33.3%	0.00%	**13.40%**

Model H's one-hop scores sit within 2.6 points of chance, and `numeric3` within **0.1** points. The two-hop family is the only exception — and it is *below* chance for both models.

9.4 Why the two-hop task scores below chance

This is the best answer in the whole project. It is a *diagnosed* failure, not a shrug.



Say this out loud

"The chain is stated as $A > B$, $B > C$, so the two extremes are always A and C and the middle entity is **never** the correct answer. Model H answers with the middle entity 72.6% of the time and with A **0.0%** of the time.
B is the only name mentioned *twice*. The model is selecting on salience instead of following the inequality. That is exactly why it scores below random guessing — random would at least hit A and C sometimes."

9.5 One-hop answers show a recency bias

Model H family	Answered first	Answered second	Gold split
`compare2`	30.8%	**62.0%**	48 / 52
`numeric2`	35.0%	**65.0%**	50 / 50

Against a balanced gold distribution, Model H prefers the *second-named* entity roughly two to one. The failures show it plainly — each of these is the model answering with the most recently mentioned name:

Example: Model H recency failures

ध्यान दीजिए: नवीन वरुण से अमीर है। अमीर कौन है?
→ model said वरुण, correct answer नवीन

ध्यान दीजिए: अरुण नवीन से बड़ा है। बड़ा कौन है?
→ model said नवीन, correct answer अरुण

9.6 Model L is much less grounded in the prompt

Answered with a name not in the question	Model H	Model L
`compare2`	7.2%	**19.2%**
`numeric2`	0.0%	**25.0%**
`numeric3`	1.0%	**21.2%**
`transitive3`	2.0%	**28.8%**

Model L emits a memorised *training* name — repeatedly सरिजना — instead of one of the candidates in front of it. Model H does this too, but far less (महेश).

Both *pretrained* models do it 100% of the time (99.6% for H on `transitive3`).

Say this out loud

"Finetuning moved Model H almost entirely onto the prompt and Model L only about three-quarters of the way. This is the single clearest resource-tier difference in the phase — and the held-out name pools are what make it measurable at all. Without disjoint pools this behaviour would have scored as partial success."

9.7 Successes

The models do get questions right, including ones needing the direction flip:

Example: Correct answers

ध्यान दीजिए: प्रवीण वरुण से हल्का है। भारी कौन है? → वरुण ✓

ध्यान दीजिए: वरुण प्रवीण से गरीब है। अमीर कौन है? → प्रवीण ✓

ध्यान दिनुहोस्: नरि कल्पनाभन्दा धनी छ। धनी को हो? → नरि ✓

Be honest about these

Given the positional biases above, these are best read as the positional habit *coinciding* with the correct answer, rather than as evidence of genuine comparison. Say that yourself before the examiner says it.

Chapter 10

Phase 3 — Attention After Finetuning

10.1 The method, and why it is fair

In my project: Three controls

- Both checkpoints run on the **same** prompts. Attention entropy depends heavily on what is being attended to, so different text would confuse a finetuning effect with a text effect.
- Prompts come from the reasoning **test** split — held-out names, held-out templates — so neither checkpoint has memorised them.
- Every prompt in a batch has the **same token count**. Mean attention distance is measured in positions and the entropy baseline is $\ln(t)$, so both grow with sequence length. Comparing a 20-token prompt against a 40-token one would show a difference that is purely arithmetic.

10.2 Results

Model	Checkpoint	Mean entropy	Mean distance	Baseline
H	pretrained	1.9102	10.303	2.9545 ($T = 46$)
H	finetuned	1.8691	10.360	
L	pretrained	1.8327	9.415	2.8061 ($T = 39$)
L	finetuned	1.7833	10.158	

Finetuning made attention **sharper and longer-range at the same time** in both models. Model L's mean distance rises 0.74 positions on a 39-token prompt — the model that most needed to learn to look at the prompt moved the most.

10.2.1 Per-head shifts

Layer means hide what actually happened. Finetuning a small model for a narrow task moves a *few* heads sharply rather than shifting every head slightly.

Model	Head	Δ entropy	Δ distance
H	L1H5	−0.2734	+1.257
H	L3H1	−0.2306	+**2.585**
H	L4H3	−0.2285	+0.869
H	L4H1	—	−1.367
L	L1H3	−0.3336	+1.836
L	L5H5	−0.3323	+2.247
L	L2H6	−0.3062	+1.412
L	L3H3	—	+**3.711**

The pattern is consistent: the heads that changed most became **sharper and longer-range simultaneously**. Middle layers (1–3) dominate the list in both models — which is where Phase 2 found the previous-token heads. Finetuning appears to have repurposed part of that machinery to look further back for a name.

Do not overclaim

The shifts are real but *small*, and accuracy did not improve.
The defensible reading: finetuning changed **where** the model looks without giving it the mechanism to *use* what it finds. It learned to reach back for a name, but not *which* name.
Also say plainly: this analysis is **correlational**. It shows which heads moved; it does not prove those heads carry the comparison. The direct test would be to ablate L3H1 or L3H3 and re-score.

Chapter 11

The Question Bank

Organised by what the Phase 1–2 examiner actually did: open the code, ask why, then make you read a figure.

11.1 On the code

Q: Walk me through the forward pass.

Answer in tensor shapes, not adjectives. See the listing on page 5 onward. The three reshapes are the part they are listening for: $(B, T, 512) \to (B, T, 8, 64) \to (B, 8, T, 64)$ and back.

Q: Show me the causal mask actually works.

"My test perturbs every token after position t and measures the change in logits at positions $\leq t$. It is **exactly 0.000e+00** — not within tolerance — because softmax$(-\infty)$ is exactly zero and $0 \times v$ is exactly zero."

Q: Why divide by $\sqrt{d_k}$?

"Dot products over 64 dimensions have standard deviation about 8. Numbers that large saturate softmax into a near one-hot distribution, and a saturated softmax has almost no gradient. Dividing by $\sqrt{d_k}$ restores unit variance and keeps softmax responsive."

Q: What is in a checkpoint?

Weights, optimizer state, scheduler state, step, tokens seen, best validation loss, the full config, and both RNG states. Resume verified across three sessions.

Q: Why a different tokenizer for each language?

"Vocabularies are never shared — that is a hard constraint. Each size was chosen from that language's own measurements: fertility and unknown rate on held-out text, traded against the embedding budget, where every extra 1,000 entries costs 512,000 parameters out of a 25M budget."

11.2 On the data

Q: How did you split train / validation / test?

"At the **document level**, with a fixed seed, roughly 96/2/2 for Hindi. Document level is the point — splitting after chunking would put two pieces of the same article in train and test, and the test perplexity would be measuring memorisation. `validate_phase1.py` confirms zero overlap."

Q: What were your cleaning steps?

NFC normalise → strip zero-width and control characters → strip pure-Latin tokens → drop malformed / too short / low-Devanagari / wrong-language → deduplicate. 95.9% character retention for Hindi. Nepali adds a perplexity filter and MinHash/LSH dedup.

Q: Why NFC specifically?

"Devanagari can be encoded precomposed or decomposed — same visible string, different bytes. Without normalising first, deduplication misses real duplicates and the tokenizer learns two pieces for one grapheme."

11.3 On architecture choices

Q: What positional encoding, and why not RoPE?

See Section 4.3. Never claim RoPE. Explain that it rotates Q and K inside attention so scores depend on $i - j$, costs zero parameters, and extrapolates — then say you train at one fixed length of 512 so that property is never exercised.

Q: What breaks without positional encoding?

"Self-attention is permutation-equivariant, so the model could not tell राम श्याम से लंबा है from श्याम राम से लंबा है — opposite meanings, identical bag of tokens. My code supports `pos_encoding: "none"` precisely so this can be measured. I did not run that ablation."

Q: Why pre-norm?

"The residual path stays clean — gradients reach layer 0 without passing through a LayerNorm at every block. Post-norm needs careful warmup to train deep stacks at all."

Q: Why weight tying?

"It saves $8{,}000 \times 512 = 4{,}096{,}000$ parameters, about 15% of Model H, that would otherwise be a second representation of the same vocabulary. Input and output token spaces are the same space."

Q: Why 6 layers?

"At $d = 512$ each block costs about 3.15M parameters. Six blocks plus embeddings lands at 23.3M, just under the ∼25M target; seven would overshoot to 26.4M. I spent the budget on depth rather than width because attention needs several rounds of composition to build multi-hop dependencies."

Q: Why AdamW rather than Adam?

"Decoupled weight decay. In Adam the L2 penalty enters the gradient and then gets divided by the second-moment estimate, so it is effectively scaled per parameter. AdamW applies decay directly to the weights. I also decay *matrices only* — biases, LayerNorm gains and embeddings set scales and offsets rather than doing linear mixing."

11.4 On training

Q: Where is dropout applied, and is it on at eval?

Four sites at $p = 0.1$: embeddings, attention probabilities, attention output after W_O, feed-forward output. **Off** at evaluation — `model.eval()`. PyTorch's inverted dropout means no rescaling is needed.

Q: Why gradient accumulation?

"`batch_size` is what fits in VRAM; `grad_accum_steps` decouples the effective batch from that. $32 \times 2 \times 512 = 32{,}768$ tokens per optimiser step, identical for both models so the token budgets match."

Q: Why fp16 and not bf16?

"The Kaggle T4 is compute capability 7.5 and has no bfloat16. fp16 needs a GradScaler because its exponent range is narrow enough for gradients to underflow; bf16 would not."

11.5 On the figures

Q: Explain the relation between temperature and repetition.

"Temperature divides the logits before softmax. Low temperature sharpens the distribution so the same token keeps winning, the model re-enters a state it has been in, and it loops — deterministically, with nothing random to break it.
My numbers: repetition goes $0.598 \to 0.212 \to 0.002 \to 0.000$ as temperature rises. And chrF++ *peaks* at $T = 1.0$ while BLEU falls monotonically, because BLEU rewards the degenerate repeated n-grams."

Q: Your perplexity gap is huge. Is Model L much worse?

"No, and bits-per-byte is why." Then derive it live: $3.3109/(0.6931 \times 8.875) = 0.5382$ versus $3.8433/(0.6931 \times 11.441) = 0.4846$. Most of the gap is tokenizer fertility.

Q: What am I looking at in this heatmap?

Upper triangle uniformly zero (the causal mask), bright first column (attention sink at position 0), tight diagonal band in early layers (local positional heads), off-diagonal structure in late layers (content-based).

11.6 On Phase 3

Q: Did finetuning teach your models to reason?

"**No.**" Say it first, then show the forced-choice column against the 41.67% chance rate, then the per-task agreement with chance.

Q: Why did you add a second metric?

"Exact match on a pretrained model measures whether it guessed the output format, and floors at zero. Forced choice among the entities the question names has a known chance rate, so the baseline is interpretable rather than an artefact."

Q: Why is the two-hop task below chance?

The middle-entity answer. 72.6% for Model H, against a gold rate of 0.0%. B is the only name mentioned twice.

Q: How did you prevent leakage in synthetic data?

Held-out names (40/10/10) *and* held-out templates (14/4/4), both asserted in code. And it paid off: both models answer test questions with training names.

Q: Did finetuning change attention?

"Sharper and longer-range at once, in both models, but small — and accuracy did not improve. The reading I can defend is that it changed where the model looks without giving it the mechanism to use what it finds."

11.7 If you genuinely do not know

Say this out loud

"I do not remember that exact line — but the file is ... and the reason it is there is ..."
Then reason out loud from what you do know.

An examiner is testing whether you understand your own system. A candid reconstruction scores far better than a confident wrong answer. The one thing to never do is claim a component you did not build — RoPE is the live example.

Chapter 12

Mistakes to Own

Bring these up yourself if the conversation goes near them. Every one of them shows engineering judgement rather than weakness.

What went wrong	What I did
Shared `lma/` package violated the per-language layout guideline	Split into two fully independent packages, `hindi/` and `nepali/`, neither importing the other. Deleted the shared package.
Kaggle's default GPU is a P100 (compute capability 6.0), which current PyTorch has dropped	Every kernel launch failed even though CUDA reported the GPU as available. Fixed by requesting `NvidiaTeslaT4` explicitly and adding a guard that raises if compute capability is below 7.0, plus a real matmul probe.
Notebook cells lost their trailing newlines	nbformat stores a cell as a list of lines and all but the last must end in `\n`. Without them Jupyter joins everything into one line and the cell is a SyntaxError. Cost a GPU session. Now the generator re-parses every cell exactly as Jupyter will join it, before pushing.
`sacrebleu` was not installed on Kaggle	Evaluation failed *after* 11 hours of training. Training was unaffected, so I built a separate evaluation notebook that re-runs scoring from the saved checkpoint.
The evaluation kernel loaded stale code	It picked up the `repo/` copy saved inside the training run's own output, so it silently evaluated an older revision. Fixed with a ranking function that prefers the `lma-code` dataset.
LR schedule restored its horizon from the checkpoint	Resuming with a larger `max_steps` pinned the learning rate at its floor. Fixed: restore progress only, take the shape from the live config.
45% gender-agreement leak in the Hindi reasoning data	Hindi adjectives inflect, so 181 of 400 examples were solvable from grammar alone. Fixed by drawing all entities in an example from one gender.

What went wrong	**What I did**
Nepali Devanagari digits were all `<unk>`	The BPE vocabulary contains no Devanagari digit and has no byte fallback — 4.8% unknown rate. Switched Nepali numerals to Arabic digits.
Checkpoint selected on too coarse a grid	Evaluating every 100 steps found its best at the first eval point. A 20-step grid moved Model L from 14.90% to 29.45% exact match.
Phase 1 reported Nepali test unknown rate as 0.0	The measured value is 0.0019. Cause: BPE without byte fallback silently drops characters it cannot represent.

Chapter 13

What I Would Do Next

Say these if asked "what are the limitations?". Having a good answer here is worth a lot.

1. **The models do not perform the task.** Forced choice is at or below chance for both. Everything in the attention chapter describes a model that learned an output format.
2. **40 training entity names is too few.** It makes memorising the answer prior easier than learning to copy from the prompt, and both models took that path — validation loss turns upward by step 60. The single most promising change would be 200+ training names, which would make the prior much less rewarding. I did not run this for want of time, not because I judged it unnecessary.
3. **The positional biases are diagnosed, not fixed.** Balancing the training distribution against answer position would test whether the two-hop failure is a data artefact or a capacity limit.
4. **The attention analysis is correlational.** Ablating L3H1 or L3H3 and re-scoring would be the direct test of whether those heads carry the comparison.
5. **`transitive3` is my only multi-hop family.** Four-entity chains would show whether the middle-entity bias is about the repeated name specifically or about chain length generally.
6. **One seed per model.** The Model H versus Model L differences are not bracketed by run-to-run variance.
7. **The bonus ablation was not run.** My code supports `pos_encoding: "none"`; I did not have GPU time to retrain and run the full evaluation suite on it.

Chapter 14

The Four Required Questions

The Phase 3 report must answer these. Know the short version of each.

14.1 How did data scale and quality differ?

By raw token count the two corpora are comparable, and Nepali is nominally *larger* — 521.4M against 486.0M on the train split. So scale is not the axis on which these models differ, which is itself useful: the project holds the token budget roughly constant and varies language, tokenizer and source quality.

Quality differs in composition. Hindi came from 704,101 documents at 30.66% manual; Nepali from 1,918,265 at 26.41% — much shorter documents, drawn more heavily from web crawl. On the train split that is **343.0 tokens per document** for Nepali against **718.9** for Hindi, a factor of 2.1.

14.2 How do LM and reasoning results compare across tiers?

They point in opposite directions, and the resolution is the interesting part.

On language modelling the tiers look far apart on perplexity (27.41 vs 46.68) and essentially equal — slightly favouring Nepali — on bits-per-byte (0.5382 vs 0.4846). Bits-per-byte is the comparable measure, and it says the two models compress their languages about equally well.

On reasoning both models fail the actual task equally: forced choice at or below chance for both. The tiers separate only on *format acquisition* and on *grounding*, where Model H is three to ten times less likely to answer with a name that is not in the question.

14.3 What tokenizer / corpus factors most affected the lower-resource model?

1. **No byte fallback.** The most consequential decision in the whole project. Model L's BPE cannot represent characters absent from its training corpus, and unrepresentable input becomes `<unk>` rather than degrading gracefully. The sharpest case: **no Devanagari digit whatsoever** in the vocabulary.

2. **Coarser tokenization.** 11.44 bytes per token against 8.88. Each token covers more text, so the same 512-token context holds about 31% more Nepali — but each prediction is correspondingly

harder, which is most of the perplexity gap and almost none of a real quality gap.

3. **Shorter, web-heavier documents.** 343.0 tokens per document against 718.9. Less long-range structure per document is a plausible contributor to Model L's weaker prompt grounding, though this project does not isolate it.

14.4 What evidence explains the observed differences?

- **Forced choice against a computed chance rate** shows the models did not learn to reason. Exact match alone would have read as +35 points.
- **Held-out entity names** show memorisation: both models answer test questions with training names, Model L four to ten times more often.
- **Position-level error analysis** shows the two-hop failure is systematic — 72.6% middle-entity selection against a 0.0% gold rate.
- **Bits-per-byte alongside perplexity** shows the H/L gap is mostly tokenizer fertility.
- **Direct vocabulary inspection** shows the Devanagari-digit gap. No aggregate metric surfaced it.
- **Identical non-embedding parameter counts** license attributing all of this to data and tokenizer rather than to model size.

Chapter 15

Final Checklist Before Walking In

1. I can state the three headline findings without notes.
2. I know I did **not** implement RoPE, and I can explain what it does and why I chose learned absolute.
3. I can name all four dropout sites and say it is off at evaluation.
4. I can derive bits-per-byte on paper: $\text{BPB} = \text{CE}/(\ln 2 \times \text{bytes/token})$.
5. I can explain why the causal-mask test gives *exactly* zero.
6. I can explain why forced choice is the honest reasoning metric and why its chance rate is 41.67%.
7. I can explain the middle-entity result and why it lands below chance.
8. I know my splits are at the **document** level and why that matters.
9. I can list the mistakes in Chapter 12 and what I did about each.
10. I know the limitations in the previous chapter and can say what I would do next.

The single most important habit

When a result looks good, say what would have made it look good *without* being good — and then say how you ruled that out.
That is the difference between a student who ran a pipeline and a student who did science. Every strong moment in this project is an instance of it: bits-per-byte against perplexity, forced choice against exact match, held-out names against memorisation, same-gender entities against the agreement leak.